

1 Benchmark-Ergebnis

Messung auf demselben Grid (SampleGrid) mit 25.000 Zeilen, 2 Warmup-Läufen und 5 gemessenen Läufen; rein in-memory (ByteArrayOutputStream), kein UI/Browser.

Engine	Median	Avg	Zeilen/s	Groesse
xlsbuilder	237,5 ms	236,9 ms	105.249	850 KB
Flowingcode	5.564,7 ms	5.582,9 ms	4.493	727 KB

xlsbuilder ist ~23x schneller

Konfiguration: Java 21, Gradle 8.9, Vaadin 24.5.3, Apache POI 5.4.0 | Ausfuehrung: ./gradlew benchmark [-Prows=N] | Klasse: ExcelExporterBenchmarkTest (@Tag("benchmark"))

2 Feature-Vergleich

2.1 Ausgabeformate

Feature	xlsbuilder	Flowingcode
Excel (.xlsx)	✓	✓
CSV	✗	✓
DOCX (Word)	✗	✓
PDF	✗	✓

2.2 Datenquelle & Filterung

Feature	xlsbuilder	Flowingcode
Exportiert alle Zeilen (ungefiltert)	✓	–
Exportiert was der User sieht (aktive Grid-Filter + Sortierung)	✗	✓
TreeGrid / hierarchische Daten	✗	✓
Streaming / out-of-core (konstanter Speicher)	✓	✗

Wichtigster funktionaler Unterschied: Flowingcode liest DataCommunicator.getFilter() und getBackEndSorting() aus - der Export entspricht exakt der aktuellen Grid-Ansicht des Nutzers. xlsbuilder stellt immer eine unbeschraenkte Query und exportiert den vollstaendigen Datenbestand.

2.3 Zelltypen & Formatierung

Feature	xlsbuilder	Flowingcode
Typisierte Zellen (INTEGER, LONG, DOUBLE, DECIMAL, BOOLEAN, DATE, DATETIME, TIME)	✓	✗
Echte Excel-Formeln (z.B. =E2*0.19)	✓	✗
Excel-Formatcode pro Spalte (#,##0.00)	✓	✓
Java DecimalFormat/DateFormat kombiniert	✗	✓
Null-Wert-Handler	✗	✓

2.4 Template & Layout

Feature	xlsbuilder	Flowingcode
Einfacher Sheet-Name	✓	✓
Template-basierter Export (.xlsx-Vorlage mit Platzhaltern)	✗	✓
Benutzerdefinierte Platzhalter im Template	✗	✓
Grid-Spalten-Footer exportieren	✗	✓
Titelzeile ueber alle Spalten (auto-merge)	✗	✓
Mehrzeilige Header / Joined Headers	✗	✓
Spalten-Reihenfolge im Export ueberschreiben	✗	✓
Sheet-Nummer waehlen (bei Multi-Sheet-Templates)	✗	✓
Auto-Size Columns	✗	✓

2.5 Spalten-Konfiguration

Feature	xlsbuilder	Flowingcode
Spalte vom Export ausschliessen	✓	✓
Export-Wert pro Spalte ueberschreiben	✓	✓
Abweichender Header-Text im Export	✗	✓
Abweichender Footer-Text im Export	✗	✓

2.6 UI-Integration

Feature	xlsbuilder	Flowingcode
Fertige Download-Buttons (auto an Grid-Footer)	✗	✓
Button-Ausrichtung konfigurierbar	✗	✓
Button deaktivieren waehrend Export	✗	✓
Tooltips auf Export-Buttons	✗	✓
Dynamischer Dateiname (via Supplier)	✗	✓
Concurrent-Download-Throttling (Semaphore)	✗	✓

2.7 Performance & Architektur

Feature	xlsbuilder	Flowingcode
Performance (25.000 Zeilen, Median)	237 ms	5.565 ms
Arbeitsspeicher-Effizienz	Streaming	Alles im RAM
Benoetigt Vaadin-Session/UI	Nein	Ja
Echte Excel-Zelltypen	Ja	Nein

3 Empfehlung

xlsbuilder	Flowingcode
• Performance zaehlt (~23x schneller) • Echte typisierte Zellen benoetigt (Datum, Boolean, Formel) • Grosse Datenmengen speicherschonend streamen • Export unabhaengig von der Grid-Ansicht (kein Filter/Sort)	• Export soll aktuelle Grid-Ansicht spiegeln (Filter, Sortierung) • Hierarchische Daten (TreeGrid) exportieren • Mehrere Formate anbieten (CSV, DOCX, PDF) • Excel-Vorlage mit Platzhaltern befuellen • Fertige UI-Buttons ohne eigenen Code

4 Technische Details

Projekt	VaadinExcelExport (github.com/MaKnoNet/VaadinExcelExport)
Stack	Java 21, Spring Boot 3.3.5, Vaadin 24.5.3
xlsbuilder	de.makno.xlsbuilder:xlsbuilder:1.0.0 (Gradle Composite Build)
Flowingcode	org.vaadin.addons.flowingcode:grid-exporter-addon:2.5.0
Apache POI	5.4.0
Benchmark	ExcelExporterBenchmarkTest (@Tag("benchmark"))
Ausfuehrung	./gradlew benchmark [-Prows=N]
Build-Tool	Gradle 8.9 mit Wrapper
Datum	4. Juni 2026